79. Getting Started with Docker

1. Problem Docker Solves

- Consider a software development team with two developers:
  - **Sarah**
  - **John**
- They work on the same web application and use a **shared codebase/development environment.**
- the application uses multiple libraries with different versions
- John updates a library on his local machine and pushes the changes.
- Sarah pulls the latest code and gets the updated library
- The updated library may:
  - Not be compatible Sarah's system.
  - This may create bugs
  - Conflict with other dependencies/environment configurations.

- This creates:
  - Dependency/version conflicts
  - Compatibility issues
  - Environmental inconsistencies
  - Difficulty in reproducing and debugging problems.
- This leads to the common situation
  - **It works on my machine, but not on yours.**
- These problems can frustrate the team and delay development.

2. Docker as the Solution

- Docker solves these problems using **containerization**
- The application is packaged into a **container**
- The container includes:
  - Application code
  - Dependencies
  - Required environmental configuration
- The container provides an **isolated environment** for the application.

- Because everything required by the application is packaged together
  - Library conflicts are reduced
  - Environment differences are reduced
  - Compatibility problems are reduced
- Team members can use the same container/image and get a consistent environment

3. What Docker Solves?

- Docker helps with:
  - Dependency management
  - Compatibility issues
  - Environmental inconsistencies
  - Version conflicts
  - Consistent application execution across different environments
  - It allows developers to package an application and its dependencies into a container

4. What is Docker?

At a high-level

- Docker is an open-source platform.
- It allows you to automate:
  - Deployment
  - Scaling
  - Management of applications
- It does this using containerization

5. What is containerization?

- Containerization is a **lightweight virtualization technology**
- It packages:
  - Application
  - Dependencies
  - Runtime Libraries
  - System Tools
- These are packaged into a standard unit called a **container**.

- A container contains everything required for an application to run.
- Containers are **portable.**
- They can be moved between systems that support Docker.
- The goal is that the same container behaves consistently across different environments.

6. What is a Virtual Machine?

- A **Virtual Machine** (VM) is essentially a computer running inside another computer.
- It is an emulation of a physical computer system.
- Example:
  - Your computer has Windows
  - You can create a VM running Ubuntu.

- VMs allow multiple operating system and their applications to run on a single physical machine
- Each VM behaves like a separate computer
- A VM has its own virtual
  - CPU
  - Memory
  - Storage
  - Operating System
- The VM uses resources allocated from the physical host machine

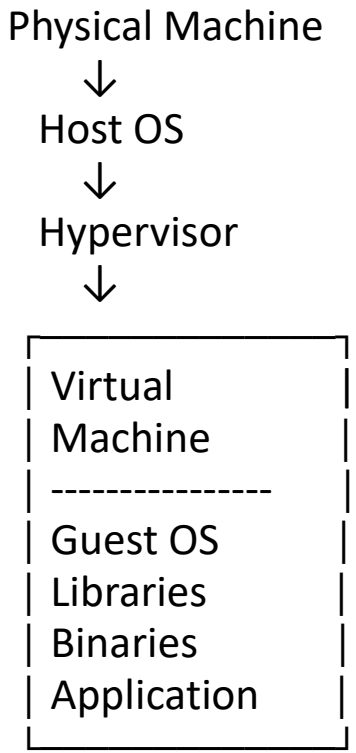
7. Virtualization Software

- VMs are created and managed using **virtualization software**
- Examples mentioned:
  - **Vmware**
  - **VirtualBox**
- These tools allow you to:
  - Create VMs
  - Allocate CPU and RAM
  - Control startup/shutdown
  - Take snapshots
  - Backup and restore VMs

- Example:
  - A 10-core CPU could be divided among multiple VM.
  - 10GB RAM could also allocated among different VMs
- Benefits:
  - Better hardware utilization
  - Ability to run multiple operating systems
  - Reduced need for separate physical machines
  - Potential cost and savings.

8. Virtual Machine Architecture

- The architecture is roughly :-



- **Host OS** = operating system installed on your physical computer.

Example: Windows

- **Hypervisor** = software that enables creation and management of VMs
- **Guest OS** = operating system running inside the VM

Example: Linux/Ubuntu

Each VM has its own guest operating system and required libraries/ binaries.

- Multiple VMs can run different operating systems and application on one physical machine.

9. Docker Architecture

- Docker works differently :-



- Docker uses a **Docker Engine**
- The Docker Engine manages containers
- A Docker container contains:
  - Application
  - Required Libraries
  - Required Binaries
- **It does not contain a complete guest operating system like a VM does.**
- this makes containers much more lightweight
- Multiple containers can run on the same machine.

10. Docker Containers vs Virtual Machines

Aspect	Virtual Machine	Docker Container
Size	Large	Lightweight
Resources	Resource-intensive	Resource-efficient
Startup	Slower	Very fast
OS	Has its own guest OS	No separate guest OS
CPU/Memory	Uses more resources	Uses fewer resources
Isolation	Strong isolation	Isolated but shares host OS kernel
Portability	Portable, but OS compatibility matters	Highly portable
Scaling	Create new VMs	Create more containers
Development setup	Slower	Faster
Deployment	More overhead	Easier/faster
Ecosystem	VM-specific tools	Docker ecosystem

11. Most Important concept to Remember

VM:

```
Computer
   ↓
Host OS
   ↓
Hypervisor
   ↓
VM
  ├── Guest OS
  ├── Libraries
  ├── Binaries
  └── Application
```

Docker:

```
Computer
   ↓
Host OS
   ↓
Docker Engine
   ↓
Container
  ├── Libraries
  ├── Binaries
  └── Application
```

Key Difference

- VM virtualizes a complete computer/operating system, whereas a Docker container packages the application and what it needs to run while sharing the host OS Kernel.
- Therefore, according to the lecture, Docker container are generally
  - Smaller
  - Faster
  - More resource-efficient
  - Easier to scale
  - Easier to deploy
  - Highly portable